

Chapter 5 — React Query

(Services/crud.queries.ts)

File: `src/features/react-crud-v4/Services/crud.queries.ts` **One job:** Wrap service functions in React Query hooks. Handle caching, loading states, error states, and cache invalidation after mutations.

5.1 Why React Query?

Before React Query existed, developers managed server data with `useState` + `useEffect`:

```
// The old way – painful and error-prone
function MyComponent() {
  const [data, setData] = useState(null);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState(null);

  useEffect(() => {
    setIsLoading(true);
    getItems()
      .then(data => { setData(data); setIsLoading(false); })
      .catch(err => { setError(err); setIsLoading(false); });
  }, []);
}
```

Problems with this approach:

1. **No caching** — every component mount triggers a new network request, even if the data was just fetched
2. **No deduplication** — if two components need the same data, they make two separate requests
3. **No background refetching** — data goes stale silently
4. **No invalidation** — after a mutation, you have to manually refetch everywhere
5. **Race conditions** — if two requests are in flight and the first one arrives after the second, you show stale data
6. **Boilerplate** — `isLoading`, `isError`, `data` state in every component that fetches data

React Query solves all of these. It is a **server state management library** — it manages the lifecycle of data that lives on the server.

5.2 The Core Concepts

Query Keys

React Query identifies cached data by a "query key" — an array of values.

```
queryKey: ["getStudentsAttendanceSummary"]
queryKey: ["getStudentsAttendanceSummary", { year: "10" }]
queryKey: ["getResponseActivities"]
```

- Two components using the same key share the same cached data — no duplicate requests
- Keys are hierarchical: ["getStudentsAttendanceSummary"] is the parent of ["getStudentsAttendanceSummary", { year: "10" }]
- Invalidating a parent key invalidates all child keys

The Cache

React Query maintains an in-memory cache. When a query is first called, it fetches from the server and stores the result. Subsequent calls with the same key return the cached data instantly (no network request).

Stale Time

Data in the cache is either "fresh" or "stale". Fresh data is returned from cache without re-fetching. Stale data is returned from cache BUT a background re-fetch is triggered.

Default `staleTime` is 0 — data is immediately stale after fetching. This means every component mount triggers a background re-fetch (the user sees cached data instantly, then the fresh data replaces it).

For reference data that rarely changes (years, categories), we set `staleTime: 5 * 60 * 1000` (5 minutes) — no re-fetch for 5 minutes.

5.3 The Real File — Line by Line

Imports

```
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import type { AxiosError } from "axios";
import { toast } from "react-toastify";
```

useQuery — for GET operations (reading data). Returns { `data`, `isLoading`, `isFetching`, `isError`, `error` }.

useMutation — for POST/PUT/DELETE (writing data). Returns { `mutate`, `mutateAsync`, `isPending`, `isError`, `error` }.

useQueryClient — gives access to the query cache instance. Used to invalidate queries after mutations.

import type { AxiosError } — **import type** imports only the TypeScript type, not runtime code.

AxiosError is the error type Axios throws when a request fails (non-2xx status, network error, etc.). We use it as the generic error type in **useQuery** and **useMutation**.

toast — shows notification popups. `toast.success("...")` = green popup. `toast.error("...")` = red popup.

Query Keys

```
const QUERY_KEYS = {
  items: ["getStudentsAttendanceSummary"] as const,
  categories: ["getResponseActivities"] as const,
  years: ["getYearsRetrieve"] as const,
};
```

Why a `QUERY_KEYS` object? If you write query keys as string literals in multiple places, a typo creates a different key and the cache doesn't work:

```
// BAD – typo creates a different key, invalidation doesn't work
useQuery({ queryKey: ["getStudentsAttendanceSummary"] });
queryClient.invalidateQueries({ queryKey: ["getStudentsAttendanceSummry"] }); //
typo!
```

```
// GOOD – single source of truth, TypeScript catches typos
useQuery({ queryKey: QUERY_KEYS.items });
queryClient.invalidateQueries({ queryKey: QUERY_KEYS.items }); // same reference
✓
```

as const — makes the array values readonly literal types. Without **as const**, TypeScript infers `string[]`. With **as const**, TypeScript infers `readonly ["getStudentsAttendanceSummary"]`. This allows TypeScript to catch typos in query key references at compile time.

Why not export `QUERY_KEYS`? It's only used within this file. Keeping it private prevents other files from depending on the internal key structure.

useGetItems

```
export const useGetItems = (params?: GetItemsParams) =>
  useQuery<GetItemsResponse, AxiosError>({
    queryKey: [...QUERY_KEYS.items, params],
    queryFn: () => getItems(params),
    enabled: !!params,
  });
```

useQuery<GetItemsResponse, AxiosError> — two generic type parameters:

1. **GetItemsResponse** — the type of `data` when the query succeeds. TypeScript knows `data` is `ItemDto[]`.

2. `AxiosError` — the type of `error` when the query fails. TypeScript knows `error.response?.status` is a number.

`queryKey: [...QUERY_KEYS.items, params]` — spreads the base key and appends `params`. This creates a unique key per set of params:

- `["getStudentsAttendanceSummary", undefined]` — no filter
- `["getStudentsAttendanceSummary", { year: "10" }]` — year 10 filter
- `["getStudentsAttendanceSummary", { year: "11" }]` — year 11 filter

Each is cached separately. When the user changes the year filter, React Query fetches fresh data for the new key while keeping the old year's data in cache. If the user switches back to year 10, the cached data is shown instantly.

`queryFn: () => getItems(params)` — the function React Query calls to fetch data. We wrap it in an arrow function so `params` is captured from the closure at fetch time, not at hook definition time.

Why the arrow function wrapper? Without it:

```
queryFn: getItems(params) // WRONG - calls getItems immediately, passes the
Promise as queryFn
queryFn: getItems          // WRONG - passes getItems without params
queryFn: () => getItems(params) // CORRECT - React Query calls this function when
it needs data
```

`enabled: !!params` — `!!params` converts `params` to a boolean:

- `undefined` → `false` (query is disabled — no fetch)
- `{}` → `true` (query is enabled — fetch)
- `{ year: "10" }` → `true` (query is enabled — fetch)

When `enabled: false`, React Query does NOT fire the query. This implements the **"Search button" pattern**: data is only fetched after the user fills in filters and clicks Search. Without this, the query would fire immediately on page load with no filters, loading potentially thousands of records.

`useGetCategories` and `useGetYears`

```
export const useGetCategories = () =>
  useQuery<GetCategoriesResponse, AxiosError>({
    queryKey: [...QUERY_KEYS.categories],
    queryFn: getCategories,
    staleTime: 5 * 60 * 1000,
  });

export const useGetYears = () =>
  useQuery<GetYearsRetrieveResponse, AxiosError>({
    queryKey: [...QUERY_KEYS.years],
    queryFn: getYears,
```

```
    staleTime: 10 * 60 * 1000,  
  });
```

staleTime: 5 * 60 * 1000 — 5 minutes in milliseconds. Data is considered "fresh" for 5 minutes. During this window, React Query returns cached data without re-fetching, even if the component unmounts and remounts.

Why **staleTime** on categories and years but not items?

- Categories (activities) and years are **reference data** — they change rarely (maybe once a year when new activities are added)
- Student attendance data changes frequently — when users record activities, the list must be fresh
- Setting **staleTime** on reference data prevents unnecessary network requests every time the user opens the modal

queryFn: getCategories — no arrow function wrapper needed here because **getCategories** takes no parameters. We can pass the function reference directly.

useCreateItem

```
export const useCreateItem = () => {  
  const queryClient = useQueryClient();  
  
  return useMutation<CreateItemResponse, AxiosError, CreateItemRequest>({  
    mutationFn: createItem,  
    onSuccess: async () => {  
      toast.success("Activity recorded.");  
      await queryClient.invalidateQueries({ queryKey: QUERY_KEYS.items });  
    },  
    onError: () => {  
      toast.error("Unable to record activity. Please try again.");  
    },  
  });  
};
```

useQueryClient() — returns the shared **QueryClient** instance. This is the cache manager. You need it to invalidate queries after mutations.

useMutation<CreateItemResponse, AxiosError, CreateItemRequest> — three generic type parameters:

1. **CreateItemResponse** — type of **data** on success
2. **AxiosError** — type of **error** on failure
3. **CreateItemRequest** — type of the argument to **mutation.mutate(payload)** — TypeScript checks you pass the right shape

mutationFn: createItem — the function to call when **mutation.mutate(payload)** is invoked. React Query calls this, handles the Promise, and triggers **onSuccess** or **onError**.

onSuccess: `async () => { ... }` — called after the mutation succeeds. Two things happen:

1. `toast.success("Activity recorded.")` — shows a green notification
2. `await queryClient.invalidateQueries({ queryKey: QUERY_KEYS.items })` — marks the items cache as stale

Why `await` on `invalidateQueries`? Without `await`, the modal might close before the re-fetch completes, causing the table to briefly show stale data. With `await`, the re-fetch is triggered and the cache is updated before `onSuccess` returns.

Why does invalidating the cache update the table? When the cache is invalidated, React Query marks the data as stale. Any component currently subscribed to that query key (i.e., `useGetItems` is mounted) immediately triggers a background re-fetch. When the re-fetch completes, the component re-renders with the new data. The table updates automatically — no manual refresh needed.

onError: `() => { toast.error("...") }` — called if the mutation throws. Shows a red notification. React Query handles the error state internally — `mutation.isError` becomes `true`, `mutation.error` contains the `AxiosError`.

useDeleteItem

```
export const useDeleteItem = (options?: { onDeleted?: () => void }) => {
  const queryClient = useQueryClient();

  return useMutation<void, AxiosError, DeleteItemRequest>({
    mutationFn: deleteItem,
    onSuccess: async () => {
      toast.success("Activity deleted.");
      await queryClient.invalidateQueries({ queryKey: QUERY_KEYS.items });
      options?.onDeleted?.();
    },
    onError: () => {
      toast.error("Unable to delete activity. Please try again.");
    },
  });
};
```

options?: { onDeleted?: () => void } — an optional callback. The `DeleteConfirmModal` passes `onDeleted: handleDeleteCompleted` which clears the `pendingDeleteIds` state after deletion. This is a clean way to add optional post-deletion behavior without coupling the hook to the component.

options?.onDeleted?.() — optional chaining. If `options` is undefined, stop. If `options.onDeleted` is undefined, stop. If it exists, call it. This is safe even when no callback is provided.

useMutation<void, AxiosError, DeleteItemRequest> — `void` as the success type because the delete endpoint returns nothing useful.

5.4 The URL Param Override Pattern

This is an advanced pattern used when a feature needs to be configurable via URL parameters (e.g., for deep linking or testing).

```
export const useGetSettingsQuery = () => {
  const [searchParams] = useSearchParams();

  return useQuery({
    queryKey: QUERY_KEYS.settings,
    queryFn: Services.getSettings,
    staleTime: STALE_TIME,
    select: (data): FeatureSettings => ({
      showWidget: parseBoolParam(searchParams.get("showWidget")) ??
data.showWidget,
      nameFormat: parseEnumParam(searchParams.get("nameFormat"), VALID_FORMATS) ??
data.nameFormat,
    }),
  });
};
```

select — a React Query option that transforms data after it arrives from the API. It runs every time the data is accessed, with the raw API response as input.

Why override in **select instead of the page hook?** Every component calling `useGetSettingsQuery()` automatically gets the URL-overridden values. No prop drilling, no extra `useState`, no `useEffect`. If you override in the page hook and pass it as a prop, you have to thread it through every component that needs it — and if a new component needs it, you have to add another prop.

`parseBoolParam(searchParams.get("showWidget")) ?? data.showWidget` — priority chain:

1. URL param (if valid) → use it
2. API result → fall back to it

`??` is the nullish coalescing operator: use the right side only if the left is `null` or `undefined`.

5.5 What `useQuery` Returns

```
const {
  data,           // The cached data (undefined while loading)
  isLoading,     // true only during the FIRST fetch (no cached data yet)
  isFetching,    // true during ANY fetch (including background refetches)
  isError,       // true if the last fetch failed
  error,         // The AxiosError (null if no error)
  isSuccess,     // true if the last fetch succeeded
  refetch,       // Function to manually trigger a re-fetch
} = useGetItems(serverParams);
```

isLoading vs isFetching:

- `isLoading` — `true` only when there is no cached data AND a fetch is in progress. Use this to show a skeleton (the table is empty, show placeholder rows).
- `isFetching` — `true` whenever a fetch is in progress, even if cached data is already displayed. Use this to show a subtle loading indicator (spinner in the corner) while the table shows stale data.

In `ItemTable.tsx`:

```
const showSkeleton = isLoading || isFetching;
```

We show the skeleton for both cases — the initial load AND background refetches. This is a UX decision: we prefer showing the skeleton over showing stale data with a spinner.

5.6 What `useMutation` Returns

```
const mutation = useCreateItem();

// Call the mutation:
mutation.mutate(payload);           // fire and forget
mutation.mutateAsync(payload);      // returns a Promise – can await it

// State:
mutation.isPending  // true while the mutation is in flight
mutation.isError    // true if the mutation failed
mutation.isSuccess  // true if the mutation succeeded
mutation.error       // the AxiosError (null if no error)
mutation.data       // the response data (undefined if not yet succeeded)

// Reset state:
mutation.reset();    // clears isPending, isError, isSuccess, data, error
```

`mutate` vs `mutateAsync`:

- `mutation.mutate(payload)` — fire and forget. Errors are handled by `onError`. Cannot be awaited.
- `mutation.mutateAsync(payload)` — returns a Promise. Can be awaited. Errors propagate to the caller (must be caught).

In `ItemModal.tsx`, we use `mutation.mutate(request, { onSuccess: onClose })` — the `onSuccess` callback closes the modal after the mutation succeeds. This is cleaner than `mutateAsync` + `await` + `onClose()` for simple cases.

5.7 Common Mistakes

✗ Missing `enabled: !!params` on search-triggered queries


```
// WRONG – fires on page load with no filters
useQuery({ queryKey: [...QUERY_KEYS.items, params], queryFn: () =>
  getItems(params) });

// CORRECT – only fires after user clicks Search
useQuery({ queryKey: [...QUERY_KEYS.items, params], queryFn: () =>
  getItems(params), enabled: !!params });
```

✗ Not awaiting `invalidateQueries`

```
// WRONG – modal closes before re-fetch completes, table shows stale data
onSuccess: () => {
  toast.success("Created.");
  queryClient.invalidateQueries({ queryKey: QUERY_KEYS.items }); // missing await
},

// CORRECT
onSuccess: async () => {
  toast.success("Created.");
  await queryClient.invalidateQueries({ queryKey: QUERY_KEYS.items });
},
```

✗ Hardcoding query keys as strings

```
// WRONG – typo creates a different key, invalidation silently fails
useQuery({ queryKey: ["getStudentsAttendanceSummary"] });
queryClient.invalidateQueries({ queryKey: ["getStudentsAttendanceSummry"] }); //
typo!

// CORRECT – single source of truth
useQuery({ queryKey: QUERY_KEYS.items });
queryClient.invalidateQueries({ queryKey: QUERY_KEYS.items });
```

✗ Calling `useQueryClient` outside a component or hook

```
// WRONG – useQueryClient is a React hook, must be called inside a component/hook
const queryClient = useQueryClient(); // outside component – throws

// CORRECT – inside the hook function
export const useCreateItem = () => {
  const queryClient = useQueryClient(); // inside hook ✓
  return useMutation({ ... });
};
```

✗ Forgetting `staleTime` on reference data

```
// WRONG – re-fetches categories on every modal open (unnecessary network requests)
export const useGetCategories = () =>
  useQuery({ queryKey: QUERY_KEYS.categories, queryFn: getCategories });

// CORRECT – categories rarely change, cache for 5 minutes
export const useGetCategories = () =>
  useQuery({ queryKey: QUERY_KEYS.categories, queryFn: getCategories, staleTime: 5
    * 60 * 1000 });
```

Next: [Chapter 6 — Mapper and Context](#)